

Feature Extraction Choices for Machine-Learning Sentiment Analysis: A Reproduction-Style Study with SMOTE and Random Forest on Twitter US Airlines and Amazon Musical Instruments Reviews

Author: Mohammadmahdi Bababeyk

Affiliation: Malekashtar University of Technology

Abstract

Feature extraction is a decisive step in machine-learning (ML) sentiment classification because it determines what linguistic signals become available to the classifier. Inspired by a recent comparative study, this project evaluates six feature extraction / word representation methods—Bag of Words (BOW), TF-IDF, N-grams (1,2), Hashing Vectorizer, Word2Vec, and GloVe—under a controlled pipeline using a fixed Random Forest classifier and SMOTE for class imbalance. Experiments were conducted on two widely used datasets (Twitter US Airlines and Amazon Musical Instruments Reviews). After text cleaning and normalization (URL/mention removal, stopword filtering, stemming, and lemmatization), data were split 70/30 into train/test with stratification; SMOTE was applied only to the training set. To avoid memory issues, each dataset was capped at 3,000 samples and vector-space methods were limited to 5,000 features; GloVe was used in a lightweight 50-dimensional setting. Results show that on Twitter, BOW achieved the best performance (Accuracy=0.7241, F1=0.7292), while on Amazon, Hashing Vectorizer performed best (Accuracy=0.7900, F1=0.7882). Embedding-based representations (Word2Vec, GloVe) were less effective in this implementation, likely due to simple sentence pooling (mean of word vectors), limited dataset size, and embedding dimensionality constraints. We also discuss differences from the reference paper and provide practical guidance for figure inclusion and further improvements.

Keywords: Sentiment Analysis, Feature Extraction, TF-IDF, Bag of Words, Hashing Vectorizer, Word2Vec, GloVe, SMOTE, Random Forest

1. Introduction

Sentiment analysis aims to classify subjective polarity in text (e.g., positive/negative/neutral). In classical ML pipelines, model quality depends heavily on how text is transformed into numeric vectors. Different representations capture different signals: raw frequency (BOW),

discriminative weighting (TF-IDF), limited word order (n-grams), scalable hashing (HV), and semantic similarity (embeddings such as Word2Vec and GloVe). The reference article emphasizes that feature extraction choice can strongly affect both predictive performance and computational cost, even when the classifier is kept fixed.

This project follows the same experimental spirit: keep the classifier constant (Random Forest) and compare feature extraction methods on two real-world datasets, while also measuring training and prediction time.

2. Related Work and Motivation

The core reference study systematically evaluates the same six representations using Random Forest with a 70/30 split and SMOTE to mitigate class imbalance, reporting that TF-IDF provides the highest accuracy (reported ~96% on Twitter and ~99% on Amazon in that study).

Beyond this paper, sentiment analysis literature commonly reports that n-grams and TF-IDF are strong baselines, while embeddings help more when paired with models that can exploit them (e.g., neural architectures) or when sentence representation is more expressive than simple averaging.

3. Materials and Methods

3.1 Datasets

Two datasets were used:

1. Twitter US Airlines Sentiment: tweets labeled as *positive/negative/neutral*.
2. Amazon Musical Instruments Reviews: review texts with star rating; sentiment

labels were derived as:

- rating > 3 → positive
- rating < 3 → negative
- rating = 3 → neutral

Implementation constraint: To avoid RAM crashes, the code caps each dataset at MAX_SAMPLES = 3000 via random sampling (random_state=42). After cleaning, Twitter retained 2996 samples; Amazon remained 3000.

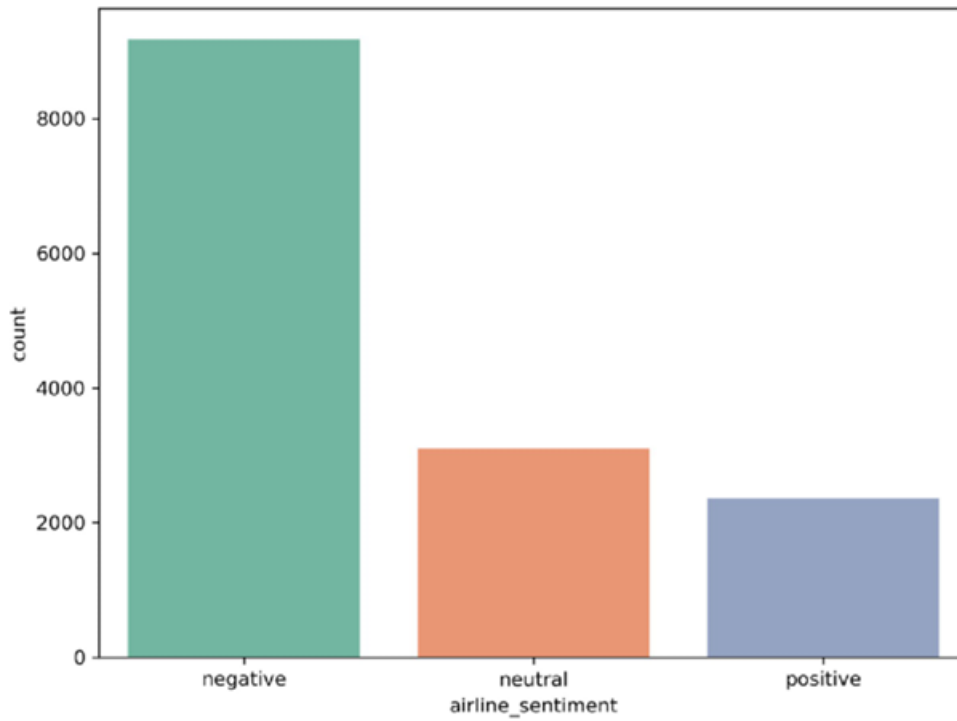


Fig1. Sentiment data distribution of the Twitter dataset

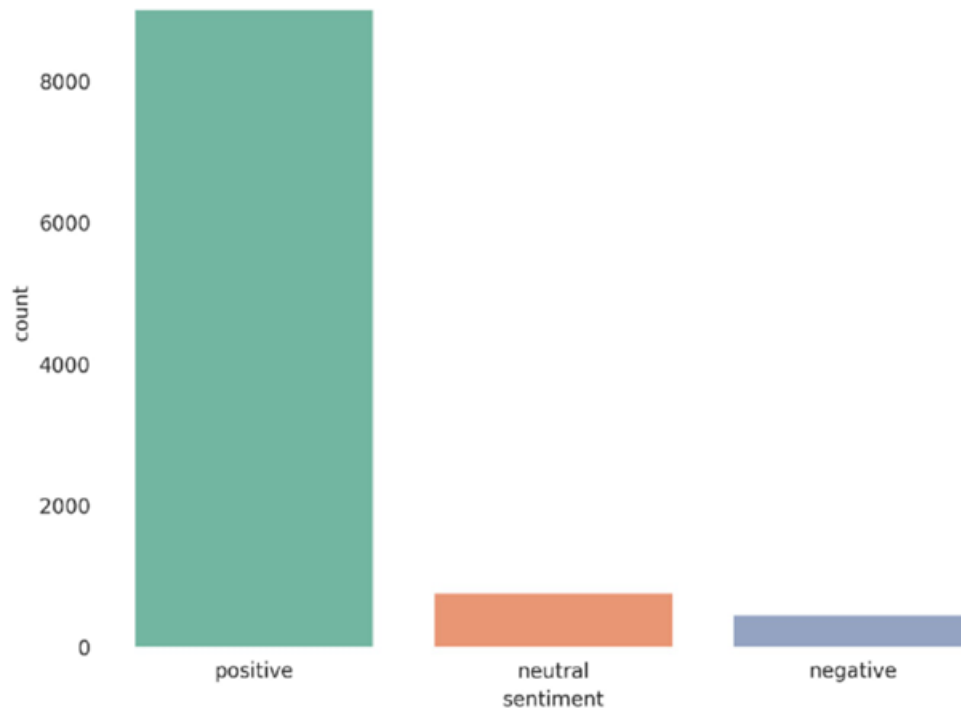


Fig2. Sentiment data distribution of the Amazon dataset

3.2 Text Preprocessing

A preprocessing pipeline was applied to standardize and denoise text:

- Lowercasing
- URL removal, mention/hashtag removal
- Punctuation stripping and removal of non-ASCII characters
- Tokenization (NLTK)
- Stopword removal
- Porter stemming + WordNet lemmatization

This is consistent with the reference paper's preprocessing choices (PorterStemmer and WordNetLemmatizer are explicitly mentioned).

3.3 Feature Extraction Methods

Six representations were compared under the same downstream classifier:

Vector-space (traditional)

1. BOW (CountVectorizer, max_features=5000)
2. TF-IDF (TfidfVectorizer, max_features=5000)
3. N-grams (1,2) (CountVectorizer with ngram_range=(1,2), max_features=5000)
4. Hashing Vectorizer (n_features=5000, alternate_sign=False)
5. Word2Vec trained on training tokens (Skip-gram, vector_size=100, window=5).
Each document vector is the mean of its word vectors.
6. GloVe loaded as glove-wiki-gigaword-50 (50D). Each document vector is the mean of word vectors, OOV → zeros.

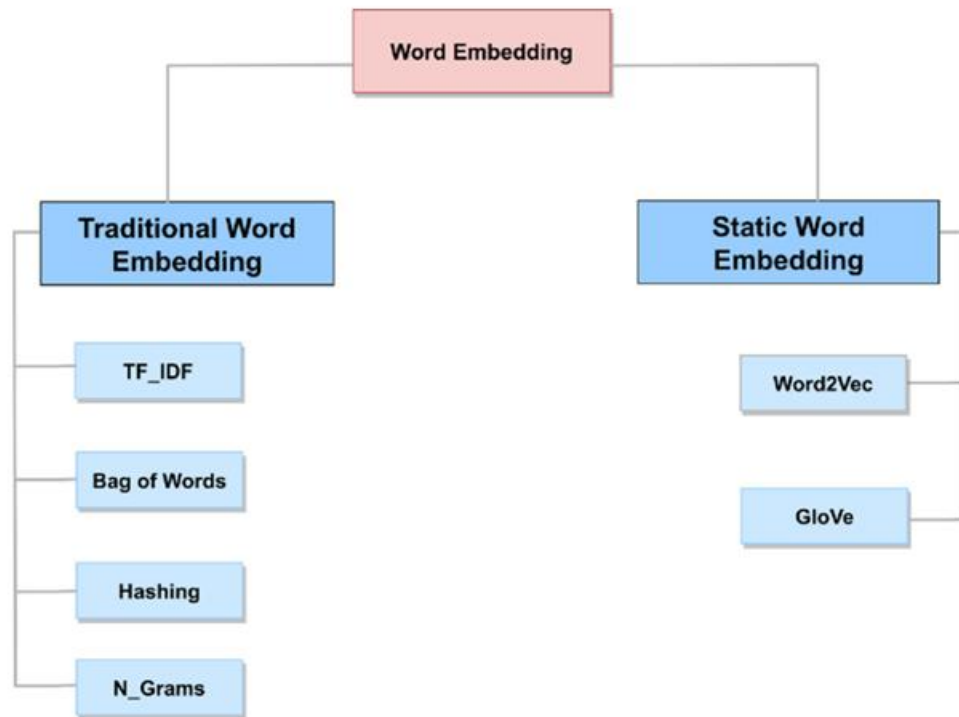


Fig3. The general structure of the word representation models

3.4 Train/Test Split and Class Imbalance Handling

- Split: 70% train / 30% test, stratified (random_state=42).
 - Imbalance handling: SMOTE applied only on the training set, producing balanced training data, while the test set remains untouched for a fair evaluation.
- The reference paper also uses SMOTE, but it states that data are split after oversampling; this methodological difference is discussed later.

3.5 Classifier and Metrics

Classifier: Random Forest with fixed settings:

- n_estimators=100, criterion=gini, max_depth=5, max_features=sqrt, random_state=42
- Metrics: Accuracy, Precision, Recall, F1 (weighted), plus training and prediction time in seconds.

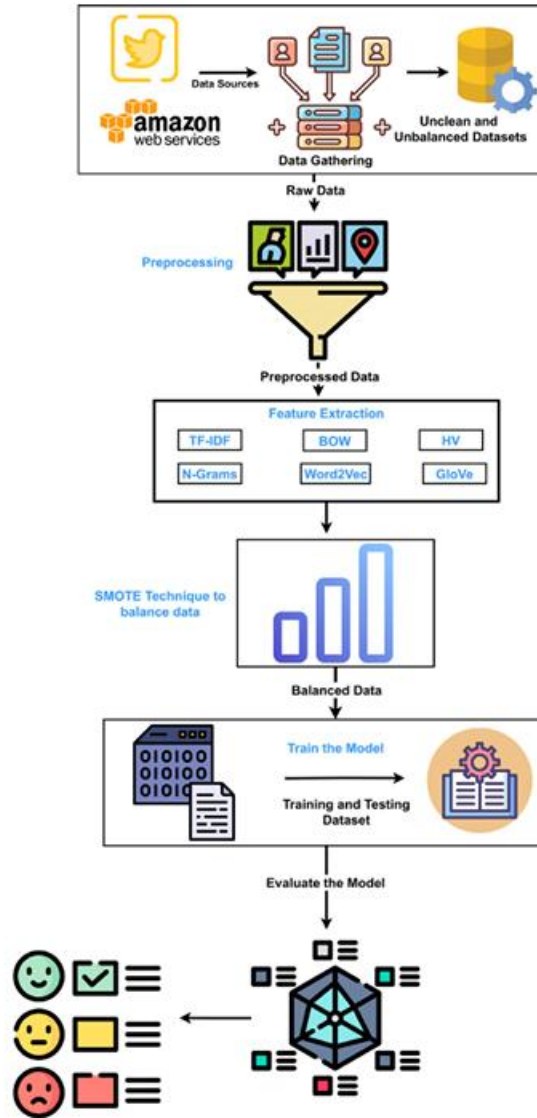


Fig4. The architecture of the proposed model for sentiment classification

4. Experimental Results

4.1 Overall Performance (Accuracy, F1)

The following values are taken directly from your experiment outputs:

Table1. Twitter (after cleaning: 2996 samples)

Method	Accuracy	F1
BOW	0.7241	0.7292
TF-IDF	0.6963	0.7110
N-Grams(1, 2)	0.6963	0.7080

Hashing Vectorizer	0.7041	0.7170
Word2Vec	0.5662	0.5770
GloVe	0.6363	0.6481

Best on Twitter: BOW (Accuracy = 0.7241, F1 = 0.7292)

Amazon (3000 samples)

Method	Accuracy	F1
BOW	0.7522	0.7700
TF-IDF	0.7611	0.7730
N-Grams(1, 2)	0.7844	0.7880
Hashing Vectorizer	0.7900	0.7882
Word2Vec	0.4778	0.5770
GloVe	0.5778	0.6609

Best on Amazon: Hashing Vectorization (Accuracy = 0.7900, F1 = 0.7882)

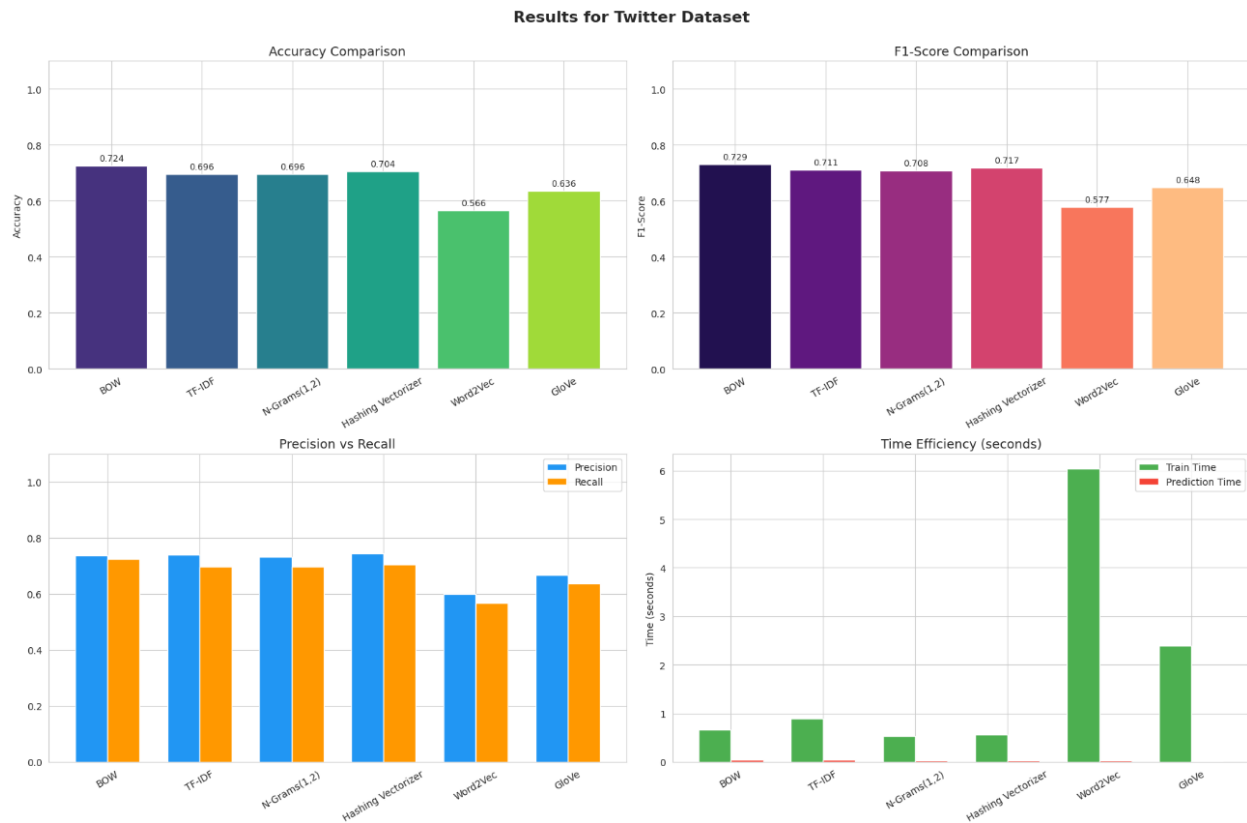


Fig5. Twitter results (Accuracy, F1, Precision vs Recall, Time)

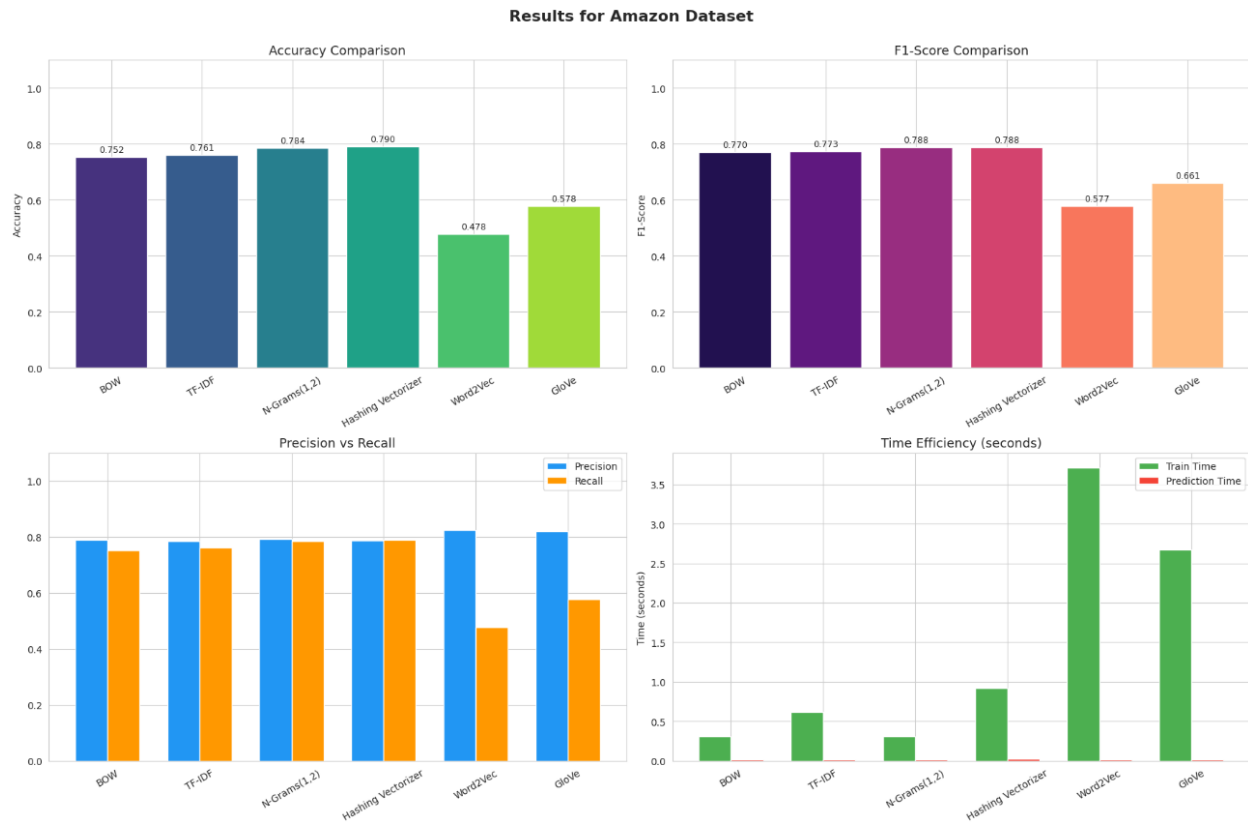


Fig6. Amazon results (Accuracy, F1, Precision vs Recall, Time)

4.2 Best Method Details per Dataset

From the printed best-method summary:

- Twitter (BOW): Accuracy=0.7241, Precision=0.7362, Recall=0.7241, F1=0.7292, Train Time=0.6552s
- Amazon (Hashing Vectorizer): Accuracy=0.7900, Precision=0.7865, Recall=0.7900, F1=0.7882, Train Time=0.9165s

5. Discussion

5.1 Why did BOW win on Twitter in this implementation?

Tweets are short and often contain direct sentiment markers (e.g., strong negative/positive terms). With a shallow but robust classifier like Random Forest, raw count signals can be sufficient—especially when vocabulary is limited to 5000 features. In addition, heavy preprocessing (stemming+lemmatization) can reduce sparsity and further favor simple count-based representations.

5.2 Why were N-grams and Hashing strong on Amazon?

Product reviews are longer and contain meaningful two-word patterns such as “high quality,” “not good,” “works great.” Bigrams can encode a small amount of order and phrase meaning, improving separability. Hashing can also perform well when the representation remains stable and collisions do not dominate; it can act as a strong baseline with controlled dimensionality (here 5000).

5.3 Why were Word2Vec and GloVe weaker here?

In this project, both Word2Vec and GloVe convert a document to a vector using mean pooling over word vectors. Mean pooling discards:

- word order,
- negation scope (“not good” vs “good”),
- emphasis and contextual composition.

Also, Word2Vec is trained only on the training split of a capped dataset (≤ 2100 documents), which can be insufficient to learn high-quality embeddings. GloVe was used in a lightweight 50D configuration for memory reasons; the reference paper reports experiments with 300D GloVe, which can capture richer semantics.

6. Comparison with the Reference Paper

The reference paper reports substantially higher accuracies and finds TF-IDF to be best overall.

My Project results differ (lower absolute accuracy; best methods vary by dataset). Likely reasons:

1. Sampling cap (3000): the reference uses the full datasets (Twitter 14640, Amazon 10261). Smaller training sets can reduce accuracy and change the ranking among feature extractors.
2. Feature cap (5000): limiting `max_features` may remove important discriminative tokens.
3. SMOTE timing difference: the reference states train/test split after oversampling (which can risk information leakage); your approach applies SMOTE only to training data, which is more standard and typically more conservative.
4. Embedding dimensionality and pooling: 50D GloVe and mean pooling can underperform compared to richer embeddings and more expressive sentence encodings.

7. Threats to Validity and Limitations

- Internal validity: results depend on fixed RF hyperparameters; another classifier (e.g., linear SVM) may change the ranking.
- Construct validity: embedding document representation via mean pooling may not reflect the true potential of Word2Vec/GloVe.
- External validity: only English datasets were considered; generalization to other domains/languages is not guaranteed.

- Resource constraints: sampling and dimensionality caps were necessary for memory stability but may reduce performance.

8. Conclusion and Future Work

This project reproduced a comparative feature-extraction evaluation for sentiment analysis using a controlled ML pipeline (SMOTE + Random Forest). On Twitter, BOW achieved the best accuracy (0.7241); on Amazon, Hashing Vectorizer achieved the best accuracy (0.7900). Embedding-based approaches underperformed in this implementation, likely due to limited training data for Word2Vec, reduced GloVe dimensionality, and simple pooling.

Future work:

1. Run on the full datasets (remove MAX_SAMPLES cap).
2. Compare 300D GloVe with 50D, matching the reference setting.
3. Improve document embeddings (TF-IDF-weighted averaging, Doc2Vec, or sentence transformers).
4. Add confusion matrices and macro-F1 for deeper class-level analysis.
5. Try additional classifiers (Linear SVM / Logistic Regression) to test robustness of conclusions.

9. References

1. Semary, N. A., Ahmed, W., Amin, K., Plawiak, P., & Hammad, M. (2024). *Enhancing machine learning-based sentiment analysis through feature extraction techniques*. PLOS ONE, 19(2), e0294968. <https://doi.org/10.1371/journal.pone.0294968>
2. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
3. Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *NeurIPS*.
4. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *EMNLP*, 1532–1543.
5. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
6. Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.
7. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
8. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. (NLTK)
9. Řehůřek, R., & Sojka, P. (2010). Software Framework for Topic Modelling with Large Corpora. *LREC Workshop*. (Gensim)

10. Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *JMLR*, 18(17), 1–5.
11. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. (TF-IDF fundamentals)